EXPERIMENT 5

Aayush Patil
D20A-56

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts.

THEORY:

1. Introduction to Solidity and Smart Contracts

Solidity is a high-level, statically typed programming language used primarily for developing smart contracts on Ethereum-compatible blockchain networks. A smart contract is a program stored and executed on a blockchain. It can contain data and functions that automatically execute according to predefined rules.

A Solidity program generally consists of a **pragma statement**, **contract declaration**, **state variables**, **functions**, **modifiers**, **constructors**, and other programming constructs.

Example structure:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.18;

contract Example {
 uint public value;

 constructor() {
 value = 10;
 }

 function getValue() public view returns (uint) {
 return value;
 }
}
```

The pragma statement specifies the compiler version, while the contract keyword defines the smart contract.

2. Primitive Data Types, Variables and Functions

Solidity provides several primitive data types used to store and manipulate information.

Common primitive data types

- **uint** – stores unsigned integers, such as uint256.
- **int** – stores signed integers.
- **bool** – stores either true or false.

● **address** – stores an Ethereum account or contract address.
● **string** – stores text.
● **bytes** – stores fixed-size or dynamic binary data.

Variables

Solidity variables can be classified as:

● **State variables** – stored permanently on the blockchain.
● **Local variables** – exist temporarily during function execution.
● **Global variables** – predefined blockchain-related variables such as msg.sender, msg.value, and block.timestamp.

Functions

Functions contain the executable logic of a smart contract.

Two important function types are:

pure: A pure function does not read or modify blockchain state. It performs calculations only using its inputs and local data.

```
function add(uint a, uint b) public pure returns (uint) {
 return a + b;
}
```

view: A view function can read state variables but cannot modify them.

```
function getValue() public view returns (uint) {
 return value;
}
```

Functions that modify blockchain state generally require a transaction and consume gas.

3. Inputs and Outputs to Functions

Solidity functions can receive input parameters and return output values. For example:

```
function multiply(uint a, uint b)
 public
 pure
 returns (uint)
{
 return a * b;
}
```

Here, a and b are input parameters and the function returns their product. A function can also

return multiple values:

```
function getDetails()
 public
 pure
 returns (uint, bool)
{
 return (100, true);
}
```

Inputs allow users or other contracts to provide information, while outputs allow the function to return calculated or stored results.

4. Visibility, Modifiers and Constructors

Function Visibility

Visibility determines who can access a function.

- **public** – can be called internally and externally.
- **private** – accessible only within the contract in which it is defined.
- **internal** – accessible within the contract and derived contracts.
- **external** – intended to be called from outside the contract.

Example:

```
function publicFunction() public {
}

function privateFunction() private {
}
```

Modifiers

Modifiers are reusable pieces of code used to control or modify the execution of functions. They are commonly used for access control.

Example:

```
modifier onlyOwner() {
 require(msg.sender == owner);
 _;
}
```

A function can then use the modifier:

```
function changeValue(uint newValue)
 public
 onlyOwner
{
```

```
 value = newValue;
}
```

Constructors

A constructor is executed only once, when the smart contract is deployed. It is commonly used to initialize state variables.

Example:

```
constructor() {
 owner = msg.sender;
}
```

5. Control Flow: if-else and Loops

Solidity supports standard control-flow statements.

if-else

The if-else statement is used to execute different code depending on a condition.

```
if (value > 10) {
 // execute statement
} else {
 // execute alternative statement
}
```

Loops

Solidity supports loops such as:

- for
- while
- do-while

Example:

```
for (uint i = 0; i < 5; i++) {
 // repeated operation
}
```

Loops should be used carefully in smart contracts because every operation consumes computational resources and may increase gas consumption.

6. Data Structures: Arrays, Mappings, Structs and Enums

Arrays

Arrays store multiple values of the same type in an ordered collection.

Example:

uint[] public numbers;

Arrays may be fixed-size or dynamically sized.

Mappings

Mappings store data as key-value pairs.

Example:

mapping(address => uint) public balances;

Here, an Ethereum address is associated with a numeric balance.

Mappings provide efficient lookup using keys, but they do not directly support iteration like arrays.

Structs

Structures allow multiple related values to be grouped together into a single custom data

type. Example:

```
struct Student {
 string name;
 uint marks;
}
```

A struct can therefore represent a complete record containing different types of information.

Enums

Enums define a set of predefined values.

Example:

```
enum Status {
 Pending,
 Active,
 Completed
}
```

Enums improve readability when a variable can have only a limited set of possible states.

7. Data Locations

Solidity uses three important data locations:

Storage

storage represents permanent data stored on the blockchain. State variables are generally stored in storage.

Because blockchain storage is persistent and relatively expensive, it should be used carefully.

Memory

memory stores temporary data during function execution. Data in memory is not permanently stored on the blockchain.

Example:

```
function process(string memory name) public {
}
```

Calldata

calldata is a non-modifiable data location used mainly for parameters of external functions. It is temporary and can be more gas-efficient than copying data into memory.

Example:

```
function process(string calldata name)
 external
{
}
```

Understanding data locations is important because they affect how data is stored and the gas efficiency of smart contracts.

8. Transactions: Ether and Wei

Ether (ETH) is the native cryptocurrency used on the Ethereum network. The smallest unit of

Ether is **Wei**.

1 Ether = 10^18 Wei
Using Wei allows transactions to represent very small amounts precisely.

9. Gas and Gas Price

Gas represents the computational work required to execute operations on the Ethereum

network. Every transaction that modifies blockchain state requires gas.

The amount paid for a transaction depends on factors including:

- Gas used
- Gas price

Efficient smart-contract programming is important because unnecessary computations can increase gas consumption.

10. Sending Transactions

Transactions are used to interact with smart contracts and modify blockchain state. For example, calling a function that changes a state variable requires a transaction.

Ether can also be transferred using Solidity mechanisms such as transfer() and call(). A

transaction may contain:

- Sender address
- Recipient address
- Value in Wei
- Gas-related information
- Contract function call data

When a state-changing transaction is submitted, it is processed by the blockchain and the resulting state change is recorded permanently.

11. Remix IDE

Remix IDE is a browser-based development environment used for writing, compiling, deploying, and testing Solidity smart contracts.

The main stages of smart-contract development in Remix are:

Write Solidity Code
↓
Compile Contract
↓
Select Environment
↓
Deploy Contract
↓
Interact with Functions
↓
Observe Results

For this experiment, the Solidity Basics Course in Remix is used to complete the assigned practical exercises.

The **Remix VM** provides a local blockchain environment inside Remix, allowing contracts to be deployed and tested without using the real Ethereum network.

12. Compilation and Deployment

Compilation

Compilation converts Solidity source code into machine-readable output that can be used by the Ethereum Virtual Machine (EVM).

The compiler checks the Solidity program for syntax and other compilation errors and produces information such as the contract's bytecode and ABI.

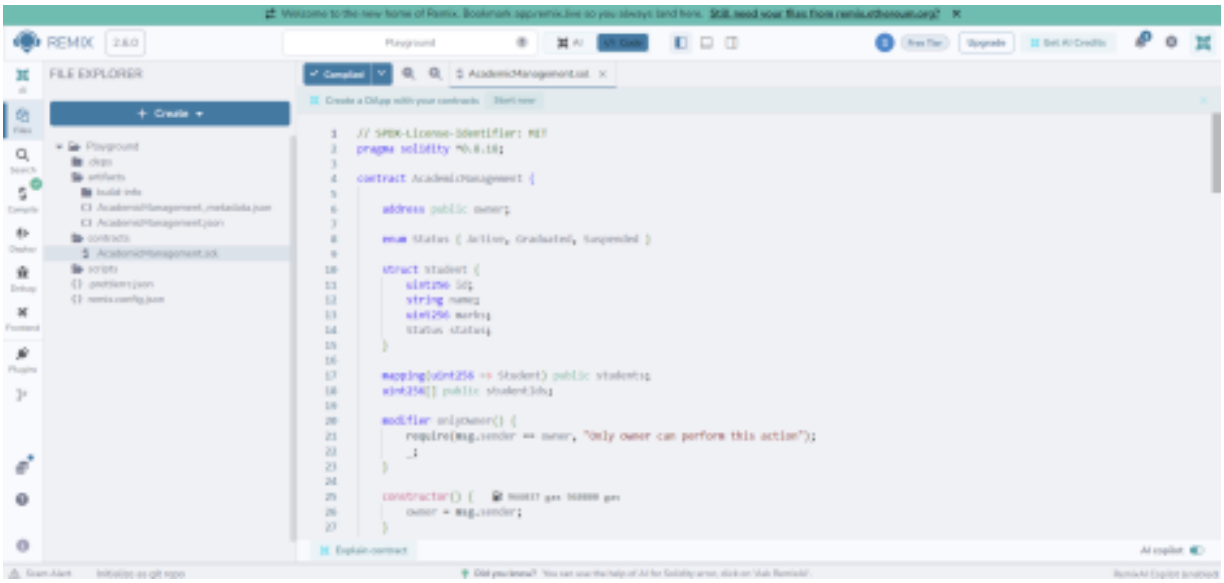
Deployment

After successful compilation, the contract can be deployed using the selected Remix environment.

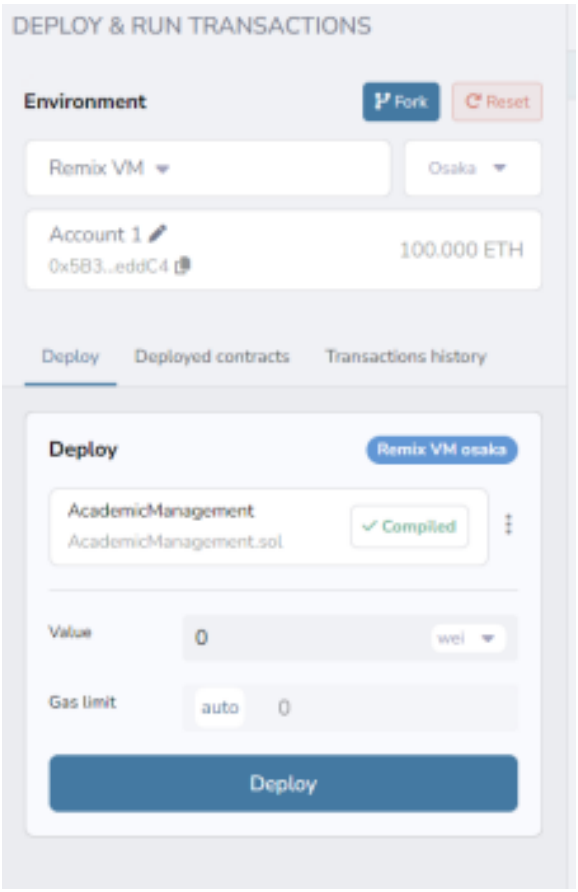
Once deployed, the contract appears under the deployed-contract section in Remix, where its functions can be called.

Functions that only read blockchain state, such as view functions, can generally be queried without creating a state-changing transaction. Functions that modify state require a transaction and consume gas.

CODE AND OUTPUT:



Student Management system code:



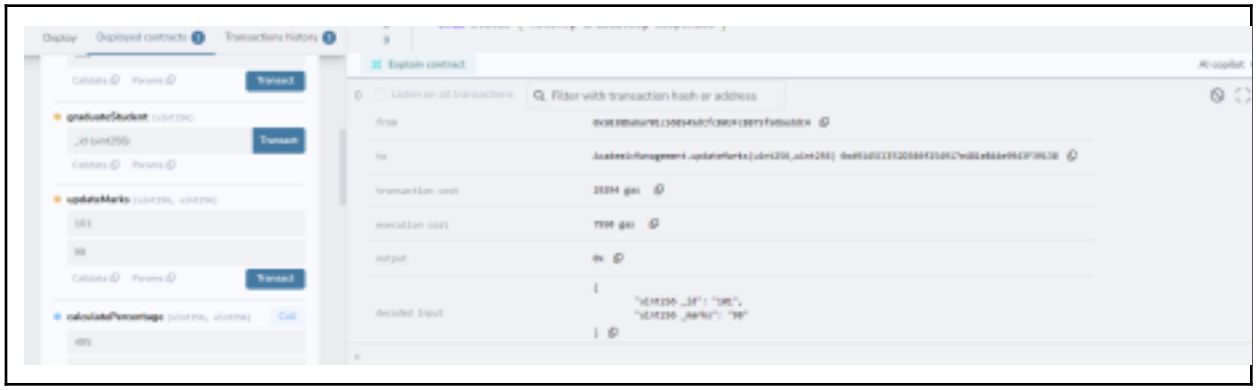


Pass/Fail:

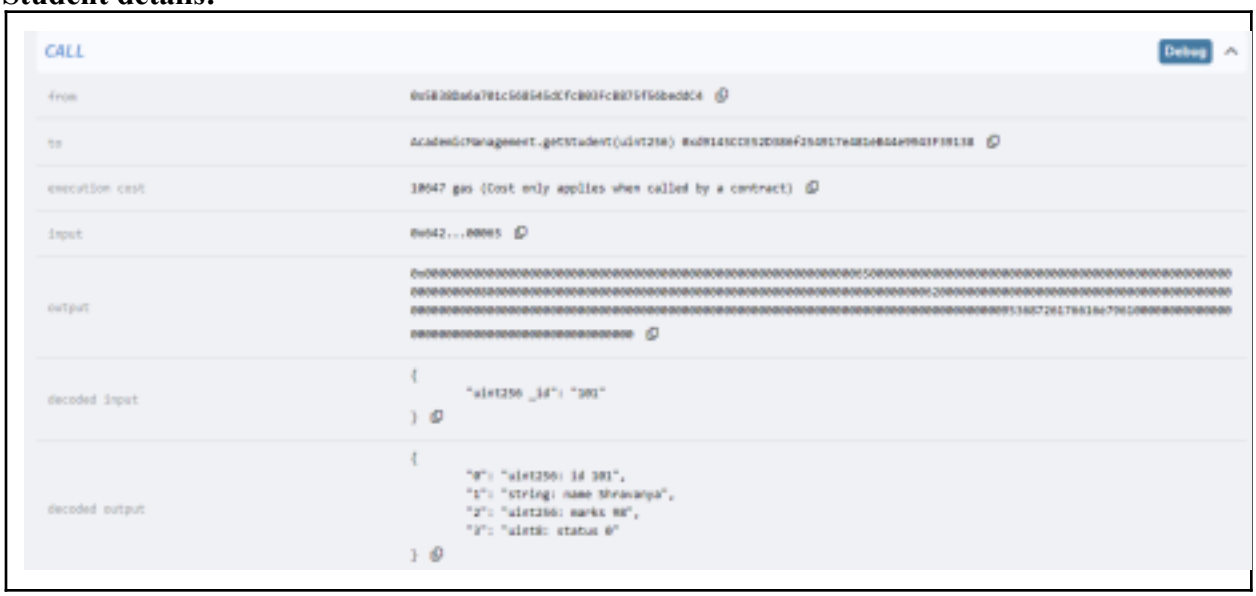


Percentage Calculation:





Student details:



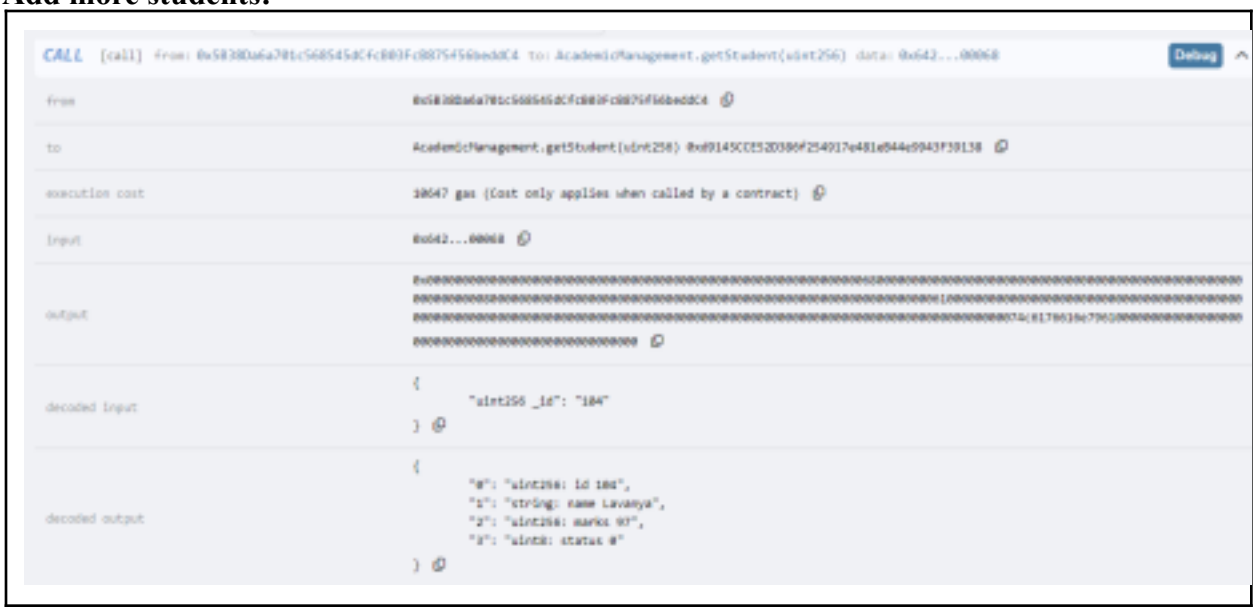




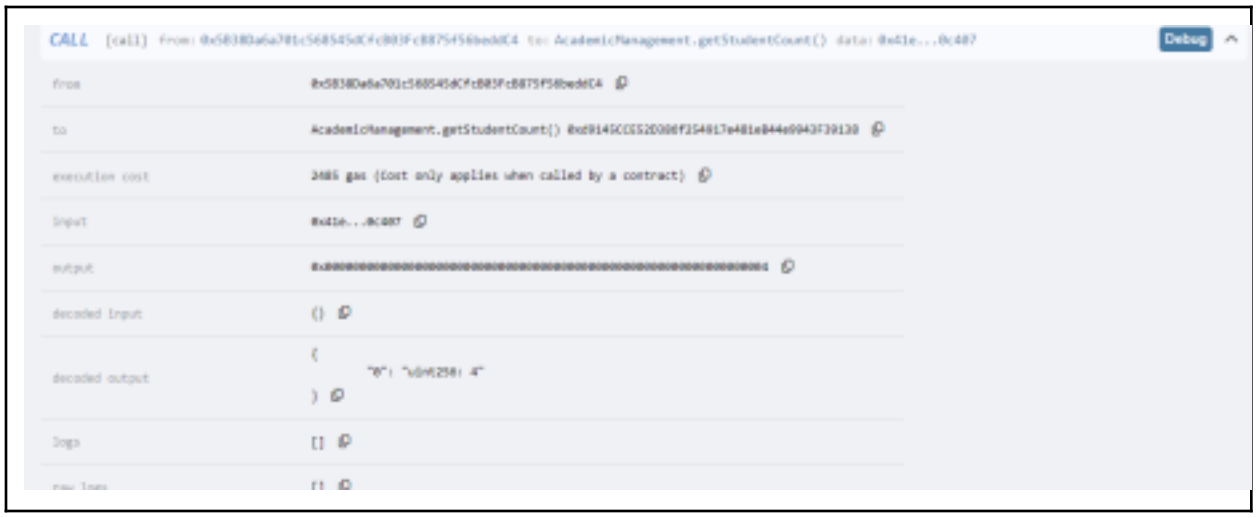
Update marks:



Add more students:



Student count:



CONCLUSION:

The **Academic Management Smart Contract** was implemented in Solidity using Remix IDE to demonstrate the basic constructs covered in this experiment. The program uses primitive data types, state variables, a Student structure, mappings and arrays to store academic records, and an enum to represent student status. Functions with view and pure keywords are used to retrieve student details and calculate percentages, while function inputs and outputs demonstrate data passing. A constructor initializes the contract owner, and an onlyOwner modifier provides access control for adding, updating, and graduating students.

The program also uses if-else control flow to determine pass or fail status, memory data location for temporary data, and require() for input validation. The contract was compiled, deployed on the Remix VM, and tested by adding students, retrieving and updating their marks, calculating percentages, checking results, and changing student status.